

Student Management System

Technical Project Report

Prepared by: Manus AI

Source: Project specification supplied by the requester

Date: 17 September 2026

Executive Summary

The Student Management System is a full-stack web application designed to help college administrators maintain student records through a centralized digital interface. The system replaces spreadsheet- and paper-based record keeping with a structured application that supports creating, viewing, searching, filtering, updating, and deleting student information.

The proposed implementation uses **React** for the frontend, **Django REST Framework** for the backend API, and **SQLite** for development-time data storage. The backend exposes RESTful endpoints for the `Student` entity, while the React single-page application provides a responsive user interface for administrators. Validation is applied at both the client and server levels, and database constraints protect unique fields such as email addresses and roll numbers.

The design also includes automated backend tests, a Postman collection for API testing, configurable environment settings, restricted local CORS access, and security guidance for future production deployment. The system therefore provides a practical foundation for a reliable student-record management platform.

1. Introduction

Colleges need an efficient way to maintain accurate student records. Paper files and unstructured spreadsheets make it difficult to search for information, enforce consistent data rules, identify duplicate records, and update information reliably. They also increase the risk of data loss and administrative errors.

The Student Management System addresses this problem through a browser-based application connected to a REST API. The application stores personal and academic information in a relational database and makes that information available through a controlled set of create, read, update, and delete operations.

The project is designed primarily for college administrators and staff who need to manage student data from a single interface. It supports common administrative tasks while keeping the frontend and backend logically separate.

2. Project Objectives

The project has four primary objectives. First, it provides a REST API that supports complete CRUD operations for student records. CRUD refers to **Create, Read, Update, and Delete**, which are the core operations required for managing stored data.

Second, it provides a responsive React interface that allows users to perform the same operations without interacting directly with the API. Third, it enforces validation on both the client and server so that incomplete, malformed, or duplicate data is rejected clearly. Fourth, it supports search and filtering by student name, email address, roll number, department, and academic year.

3. Scope of the System

The system manages the following student information:

- First name and last name
- Email address
- Roll number
- Department
- Academic year
- Phone number
- Date of admission
- Active status
- Record creation and modification timestamps

The system includes a Django administrative interface in addition to the React application. The API supports listing, retrieving, creating, updating, and deleting students. List requests can also use search terms, department filters, year filters, ordering, and pagination.

Authentication and role-based authorization are identified as future enhancements rather than current requirements. The current design is therefore suitable for development and controlled local use, but it requires additional access controls before handling sensitive student information in production.

4. Technology Stack

Application layer	Technology	Role in the system
-------------------	------------	--------------------

Frontend	React 18	Provides the single-page user interface
HTTP client	Axios	Sends requests from the frontend to the backend API
Backend	Django 5	Provides the web application framework and project structure
API layer	Django REST Framework	Exposes and validates RESTful endpoints
Database	SQLite	Stores student records during development
API testing	Postman and Django APITestCase	Supports manual and automated testing
Version control	Git	Tracks project changes

SQLite is appropriate for development and small-scale testing. A production deployment should evaluate a more robust database such as MySQL or PostgreSQL, depending on expected traffic, availability requirements, and operational standards.

5. System Architecture

The system follows a three-layer architecture. A user interacts with the React frontend through a web browser. The frontend sends JSON requests to the Django REST API using HTTP. The API processes requests through serializers, views, and the Django Object-Relational Mapper (ORM). The ORM then reads from or writes to the SQLite database.



This separation allows the frontend and backend to be developed and tested independently. It also makes it possible to replace the frontend or database later without redesigning the entire system.

6. Data Model

The central entity is the `Student` model. Each record has a unique numeric identifier. The email address and roll number are unique, which prevents the same student identity fields from being assigned to multiple records.

The first name, last name, department, year, and date of admission are required fields. The phone number is optional. The `is_active` field allows the system to distinguish active records from records that may no longer be in current use. The `created_at` and `updated_at` fields provide an audit trail for basic record timing.

Field	Data type	Required	Constraint or purpose
<code>id</code>	Integer	Yes	Primary key
<code>first_name</code>	String	Yes	Student's given name
<code>last_name</code>	String	Yes	Student's family name
<code>email</code>	String	Yes	Unique email address
<code>roll_number</code>	String	Yes	Unique institutional identifier
<code>department</code>	String	Yes	Department choice
<code>year</code>	Integer	Yes	Academic year from 1 to 4
<code>phone</code>	String	No	Optional contact number
<code>date_of_admission</code>	Date	Yes	Admission date
<code>is_active</code>	Boolean	Yes	Indicates whether the record is active
<code>created_at</code>	Date and time	Automatically generated	Record creation time
<code>updated_at</code>	Date and time	Automatically generated	Last modification time

7. Project Structure

The proposed repository separates backend and frontend responsibilities. The backend contains the Django project, application logic, serializers, views, URLs, administrative configuration, and tests. The frontend contains the React application, API client, reusable components, and public entry point.

Plain Text

```
studentms/
├── backend/
│   ├── manage.py
│   ├── requirements.txt
│   ├── studentms_backend/      # project settings, URLs, WSGI, and ASGI
│   └── students/              # models, serializers, views, URLs, admin, tests
├── frontend/
│   ├── package.json
│   ├── public/index.html
│   └── src/                   # App.js, api.js, and components
├── postman_collection.json
└── README.md
```

This organization keeps framework configuration separate from domain-specific student-management code. It also provides a clear location for automated tests and API documentation assets.

8. REST API Design

The API uses the base URL `http://127.0.0.1:8000/api` during local development. Student resources are available under the `/students/` endpoint.

Operation	HTTP method	Endpoint	Description
Create	POST	<code>/students/</code>	Creates a new student record
List	GET	<code>/students/</code>	Returns student records with search, filters, ordering, and pagination
Retrieve	GET	<code>/students/{id}/</code>	Returns one student record
Update	PUT or PATCH	<code>/students/{id}/</code>	Replaces or partially updates a record

Delete	DELETE	/students/{id}/	Removes a student record and returns a confirmation message
--------	--------	-----------------	---

The list endpoint supports query parameters such as `search`, `department`, `year`, and `ordering`. For example, a user can request students named Kumar in the Computer Science department who are in the second academic year:

Plain Text

```
GET /api/students/?search=Kumar&department=CSE&year=2
```

A typical create request contains the following JSON data:

JSON

```
{
  "first_name": "Asha",
  "last_name": "Kumar",
  "email": "asha.kumar@example.com",
  "roll_number": "CSE2024001",
  "department": "CSE",
  "year": 2,
  "phone": "+919876543210",
  "date_of_admission": "2024-06-15"
}
```

9. Validation and Error Handling

Validation is implemented at two levels. The React form validates required fields, email formatting, and phone-number formatting before submitting a request. This gives users immediate feedback and reduces unnecessary network requests.

The Django serializer provides authoritative server-side validation. It validates field values, normalizes email addresses and roll numbers, and enforces the permitted academic-year range. Database-level uniqueness constraints provide an additional safeguard for email addresses and roll numbers.

The view layer catches database `IntegrityError` exceptions. Instead of allowing a duplicate record to cause an unhandled server error, it returns a clear HTTP 400 response explaining that the submitted data conflicts with an existing record. The frontend also displays a connection error when the backend cannot be reached.

10. Installation and Execution

The project requires Python 3.10 or later, Node.js 18 or later, npm, and internet access for installing dependencies.

Backend setup

Bash

```
cd backend
python -m venv venv
source venv/bin/activate      # Windows: venv\\Scripts\\activate
pip install -r requirements.txt

python manage.py migrate
python manage.py createsuperuser # Optional, for /admin/
python manage.py runserver      # http://127.0.0.1:8000
```

Frontend setup

Bash

```
cd frontend
npm install
npm start                # http://localhost:3000
```

By default, the React application communicates with `http://127.0.0.1:8000/api`. This value can be changed through a frontend `.env` file:

Plain Text

```
REACT_APP_API_URL=<your-api-url>
```

The Django administrative interface is available at `http://127.0.0.1:8000/admin/` after a superuser has been created.

11. Testing Strategy

The backend test suite contains ten tests covering normal and error conditions. The tests address creation, retrieval, updating, deletion, invalid input, missing fields, duplicate values, and requests for nonexistent identifiers.

The automated suite can be run with the following command:

Bash

```
cd backend  
python manage.py test
```

The Postman collection provides a second method for validating the API. It covers create, read, update, and delete operations, as well as valid, invalid, missing, duplicate, and nonexistent-ID cases.

Frontend testing should confirm that the interface remains usable at different browser widths. It should also verify that the application shows an understandable error banner when the backend is unavailable. Together, automated backend tests, manual API tests, and browser-level checks provide coverage across the main system layers.

12. Security Considerations

The design avoids hard-coding secrets. The Django secret key, debug setting, and allowed hosts are read from environment variables with safe local defaults. CORS is restricted to the local frontend origin, which prevents arbitrary browser origins from making requests during development.

All database access uses the Django ORM, which provides parameterized query handling through the framework's data-access layer. These measures reduce common configuration and query risks, but they do not replace production security controls.

Before deployment, the project should set `DJANGO_DEBUG=False`, use a strong secret key, configure `DJANGO_ALLOWED_HOSTS`, and add authentication and authorization. Because student records can contain personal information, production use should also consider encrypted transport, secure credential management, access logging, backups, retention rules, and privacy requirements.

13. Challenges and Solutions

Duplicate email addresses and roll numbers are a central data-integrity risk. The project addresses this risk through unique database constraints, serializer validation, and explicit handling of `IntegrityError` exceptions. This layered approach provides both preventative validation and safe handling of race conditions or unexpected duplicate submissions.

A second challenge is maintaining a clean separation between the frontend and backend. The project solves this through CORS configuration and a shared Axios client in `api.js`. The client centralizes the API base URL and allows the frontend to be configured for different environments without changing request logic throughout the application.

14. Future Enhancements

The next major enhancement should be authentication and authorization. Different roles, such as administrators and staff members, could receive different permissions for viewing, editing, or deleting student records.

Additional improvements could include pagination controls in the user interface, CSV export, file uploads for student photographs or documents, and deployment through Docker with a production web server such as Nginx. A production database such as PostgreSQL or MySQL should also be considered when the system grows beyond local development or small-scale use.

15. Conclusion

The Student Management System provides a clear and extensible foundation for digital student-record administration. Its REST API supports complete CRUD functionality, while the React frontend gives users a practical interface for managing records. Validation at the client, serializer, and database levels improves data quality and reduces avoidable errors.

The architecture is modular, the project structure is straightforward, and the included automated tests and Postman collection support verification. The system is appropriate as a development or academic project and can be prepared for production by adding authentication, stronger deployment configuration, a production-grade database, and comprehensive privacy and security controls.

References

- [1] Student Management System project specification supplied by the requester
- [2] Django REST framework documentation
- [3] React documentation
- [4] Django documentation
- [5] SQLite documentation

Report basis: The project-specific requirements, architecture, API details, validation rules, testing scope, and security notes in this report are derived from the supplied project specification [1](#) . Framework and database names are linked to their official documentation for reference [2](#) [3](#) [4](#) [5](#) .